

STUDENT PROJECT BOOK

Object-Oriented C++ Project Book

Extended Activities & Capstone Challenges — Part 3 Companion

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

How to Use This Book

This book is a companion to Part 3 of the Introduction to C++ Programming course (Modules 20-35). It does not re-teach the lessons — read each module's README and complete its own exercise and project first. What follows here are extended, independent activities for each topic: more practice, applied to new scenarios, with no starter code and no solutions provided.

That's deliberate. By Part 3 you are learning to DESIGN, and design can only be learned by making decisions yourself — which classes, who owns what, which relationship, which guarantee. Several activities ask for a short written explanation as well as code. Treat those explanations as seriously as the code: being able to justify a design is what turns a programmer into an engineer.

Each chapter offers three activities of increasing difficulty, followed by an optional domain twist you can apply to any of them for extra practice in Electrical/Electronic Engineering or Biomedical Engineering scenarios. The final chapter is an Extended Capstone — a bigger, optional challenge that goes beyond Module 36's own capstone.

From Module 22 onwards, build every activity as a small multi-file CMake project, compile with `-Wall -Wextra` (zero warnings), and — from Module 24 onwards — run it at least once with `-fsanitize=address,undefined`. Commit each activity to your course GitHub repository in a folder such as `project-book/module-20/`, exactly as you've been committing every exercise and project so far.

Chapter 20 — OOP Principles & Object Modelling

Recap: Encapsulation keeps data and its rules together; abstraction hides how; inheritance models is-a; polymorphism gives one interface many behaviours. Noun/verb analysis turns a problem statement into candidate classes, attributes and methods; CRC cards and text UML record the design before any code is written.

Activity 1 — Canteen Design on Paper

DIFFICULTY: FOUNDATIONAL

Scenario: *A school canteen sells meals and drinks. Students pay with cash or mobile money; the canteen tracks daily sales and warns when an item is running out.*

Requirements:

- Write the problem statement above as three or four full sentences, adding any detail you think is missing.
- Produce a noun/verb table, marking each noun as class, attribute or neither with a one-line reason.
- Write a CRC card for every class you keep, and a text UML diagram with + and - members. No code yet — this activity is design only.

Activity 2 — Procedural to Object-Oriented

DIFFICULTY: CORE

Scenario: *You have inherited a 'student marks' program that stores names, three scores per student and a 'passed' flag in four parallel vectors, with free functions that update them.*

Requirements:

- Write that procedural program yourself (about 60 lines), including one function that updates the scores but forgets to update the 'passed' flag.
- Rewrite it as one or two classes in which the inconsistency is impossible.
- In a comment, list every rule the class now enforces that the procedural version could not.

Activity 3 — Spot the Missing Pillar

DIFFICULTY: CHALLENGE

Scenario: *Real designs often use some of the four pillars well and others badly.*

Requirements:

- Find (or write) a 100-200 line C++ program — your own Part 2 capstone is ideal — and write a one-page review identifying where each of the four pillars is used well, and where each is missing or misused.
- Choose the single most valuable improvement and implement it.
- Draw before and after UML diagrams of the classes you changed.

Domain Twist (optional)

EE twist: design a solar-powered borehole system (panel, inverter, battery, pump, controller) on paper with full CRC cards. Biomedical twist: design a hospital pharmacy dispensing system (prescriptions, stock, pharmacists, dispensing records).

Chapter 21 — Class Design in Depth

Recap: Initialise members in the initializer list, give built-in members default values, remove duplicated validation with delegating constructors, mark single-argument constructors explicit, and make every read-only method const. State each class's invariant and design operations that preserve it, rather than writing a setter for every member. static members belong to the class.

Activity 1 — A Robust Date Class

DIFFICULTY: FOUNDATIONAL

Scenario: *Dates appear everywhere in your track: appointments, payments, readings.*

Requirements:

- Build a Date class with an invariant (a real day in a real month, including leap years), enforced by its constructor and every method.
- Provide addDays(int), daysUntil(const Date&) const, and a const print() method, defined outside the class.
- Include a static isLeapYear(int) helper and a delegating constructor for 'the first of a given month'.

Activity 2 — Getters, Setters, or Operations?

DIFFICULTY: CORE

Scenario: *A colleague's Thermostat class has get/set methods for target, current temperature, mode and fan speed — and nothing else.*

Requirements:

- Write that class as described, then list at least three nonsense states it allows (e.g. a heating mode with a target below the current temperature).
- Redesign its public interface around operations (e.g. requestComfort(double), enterEcoMode()) and a written invariant.
- Write a short comparison explaining which design would be safer for a junior programmer to use, and why.

Activity 3 — ID Generators Without Clashes

DIFFICULTY: CHALLENGE

Scenario: *Several classes in your track need unique, readable IDs such as MP-0001, CL-0042 or DEV-007.*

Requirements:

- Write a small IdGenerator class whose static members hand out IDs per prefix (a different counter for each prefix).
- Make the IDs const members of the objects that receive them.
- Explain in a comment what would go wrong if two parts of a program each created their own counter, and how static members prevent it.

Domain Twist (optional)

EE twist: a Resistor class with E12 preferred-value validation (only standard values allowed) and a tolerance band. Biomedical twist: a BloodPressure class whose invariant requires systolic > diastolic and both within physiological limits.

Chapter 22 — Multi-File Projects & Namespaces

Recap: Headers declare and source files define. `#pragma once` protects headers. Compiling turns each `.cpp` into an object file; linking joins them — 'undefined reference' and 'multiple definition' are linker errors. Forward declarations break include cycles; namespaces prevent name clashes; CMake describes the build once.

Activity 1 — Split Your Part 2 Capstone

DIFFICULTY: FOUNDATIONAL

Scenario: *Your Module 19 capstone is probably one large file.*

Requirements:

- Split it into one header and one source file per class, inside a namespace, with a `main.cpp`.
- Add a `CMakeLists.txt` and build it both with CMake and with a single `g++` command.
- Prove the output is identical to the single-file version.

Activity 2 — Linker Error Clinic

DIFFICULTY: CORE

Scenario: *Linker errors are confusing because they don't point to a line of your code.*

Requirements:

- Deliberately create five different broken builds of a small multi-file project: a missing `.cpp`, a missing `ClassName::`, a mismatched `const`, a function body in a header, and an undefined static member.
- For each, record the exact error message and a one-sentence diagnosis.
- Turn your notes into a one-page 'linker error cheat sheet' (as a Markdown file) for future classmates.

Activity 3 — A Small Shared Library

DIFFICULTY: CHALLENGE

Scenario: *Several of your projects share helper code (formatting money, parsing CSV lines, validating ranges).*

Requirements:

- Create a utilities project in its own namespace, built by CMake as a library with `add_library`, and use it from two different executables in the same CMake project with `target_link_libraries`.
- Make sure no header contains `using namespace` and every header is self-sufficient (it compiles when included first in an empty `.cpp` file).
- Explain in a README which files are recompiled when you change one helper's `.cpp` file versus its `.h` file.

Domain Twist (optional)

EE twist: organise a units-conversion library (volts, amps, watts, kWh) into makersplace::energy::units. Biomedical twist: organise a clinical-calculations library (BMI, BSA, dose-per-kg) into makersplace::clinic::calc.

Chapter 23 — Object Lifetime, Destructors & RAI

Recap: Automatic objects die at the end of their block, dynamic objects when deleted, and destruction is always the reverse of construction. RAI ties a resource to an object: acquire it in the constructor, release it in the destructor, and make the guard an automatic object so clean-up happens on every exit path.

Activity 1 — Lifetime Prediction Puzzles

DIFFICULTY: FOUNDATIONAL

Scenario: *Accurate mental models of lifetime prevent whole categories of bugs.*

Requirements:

- Write five short programs that each mix locals, nested blocks, temporaries, members and a base/derived pair, all with tracing constructors and destructors.
- Swap programs with a classmate (or wait a day) and predict each output exactly before running it.
- Record every wrong prediction and the rule you had forgotten.

Activity 2 — A Scoped Timer

DIFFICULTY: CORE

Scenario: *Measuring how long a section of code takes is a classic RAI use.*

Requirements:

- Write a ScopedTimer class that records the time in its constructor (using <chrono>) and prints the elapsed milliseconds with a label in its destructor.
- Use it to time three different ways of filling a large std::vector, including one with an early return.
- Explain why the timer still reports correctly when the function returns early.

Activity 3 — RAI Around a C-Style API

DIFFICULTY: CHALLENGE

Scenario: *Many libraries (especially for hardware) offer C-style open/close functions that are easy to misuse.*

Requirements:

- Write a tiny simulated C-style API: int deviceOpen(const char* name), void deviceWrite(int handle, double value), void deviceClose(int handle), which prints an error if a handle is closed twice or never closed.
- Wrap it in an RAI DeviceHandle class so that every handle is closed exactly once.
- Write a function with three early returns that uses the wrapper, and show the simulated API reports no errors.

Domain Twist (optional)

EE twist: an RAI RelayGuard that energises a relay and always de-energises it.

Biomedical twist: an RAI SterileFieldSession that records when a sterile field is opened and always records its closure, even if a procedure is abandoned.

Chapter 24 — Copy Semantics

Recap: Copies happen on initialisation from another object, pass-by-value, return-by-value and insertion into containers. The default memberwise copy is shallow for owning raw pointers. A class that needs a destructor usually needs a copy constructor and copy assignment too (the Rule of Three) — or should forbid copying — but the best design follows the Rule of Zero.

Activity 1 — Count Every Copy

DIFFICULTY: FOUNDATIONAL

Scenario: Many beginner programs copy large objects without anyone noticing.

Requirements:

- Give a class a tracing copy constructor and copy assignment that count calls.
- Write ten short functions that pass, return and store the object in different ways, and predict the count for each before running.
- Rewrite the functions that copied unnecessarily so that they don't, and record the new counts.

Activity 2 — A Deep-Copying Matrix

DIFFICULTY: CORE

Scenario: Engineering calculations often use matrices whose size is only known at runtime.

Requirements:

- Build a Matrix class storing rows x cols doubles in a single new double[] array, with the full Rule of Three.
- Support at(row, col) (bounds-checked) and multiplication by another Matrix.
- Prove with AddressSanitizer that copying, assigning and self-assigning are all safe; then rewrite it with std::vector<double> and the Rule of Zero, and compare line counts.

Activity 3 — Copy-and-Swap vs Classic Assignment

DIFFICULTY: CHALLENGE

Scenario: There are two common ways to write a correct copy assignment operator.

Requirements:

- Implement the same class twice: once with the classic self-check / copy-first / free-second operator=, and once with copy-and-swap.
- Write a test that makes allocation fail part-way (hint: a constructor that throws after a set number of allocations) and show both versions leave the target unchanged.
- Write a one-page comparison: readability, safety, and performance.

Domain Twist (optional)

EE twist: a deep-copying FirmwareImage (a byte buffer) so a technician can patch a copy before flashing. Biomedical twist: a deep-copying MriSlice (a 2-D intensity grid) so a radiologist's annotations never touch the original scan.

Chapter 25 — Move Semantics

Recap: Rvalues are temporaries; moving steals their resources instead of copying them, leaving the source empty but valid. `std::move` is only a cast that grants permission to move. Mark move operations `noexcept` so `std::vector` uses them. Think about all five special members together — or, better, need none of them.

Activity 1 — Moved-From Detective

DIFFICULTY: FOUNDATIONAL

Scenario: *What exactly is left in a moved-from standard object?*

Requirements:

- For `std::string`, `std::vector<int>`, and `std::unique_ptr<int>`, move from an object and print its state afterwards.
- Then assign a new value to each moved-from object and use it normally.
- Write a short note on what you observed and why your code must not depend on the moved-from contents.

Activity 2 — A Move-Only Log File

DIFFICULTY: CORE

Scenario: *An open log file is a unique resource: it can be handed over but never duplicated.*

Requirements:

- Write a `LogFile` class (wrapping `std::ofstream`) that is move-only: copying deleted, `noexcept` moves provided.
- Write a function that creates and returns a `LogFile`, and store several in a `std::vector`.
- Show with tracing that each file is opened once and closed once, by its final owner.

Activity 3 — Measuring the Difference

DIFFICULTY: CHALLENGE

Scenario: *Claims about performance should be measured.*

Requirements:

- Build a class holding a 1,000,000-element `std::vector<double>` member, and a second version holding a raw array with only copy operations.
- Time (with `<chrono>`) pushing 200 of each into a `std::vector`, with and without `reserve()`.
- Present your measurements in a table and explain each result using what you know about moves, `noexcept` and vector growth.

Domain Twist (optional)

EE twist: a move-only `SerialConnection` to a microcontroller, handed from a 'scanner' function to a 'logger' object. Biomedical twist: a move-only `PatientRecordLock` that

must be handed from the admissions desk to the ward without ever existing in two places.

Chapter 26 — Smart Pointers & Ownership

Recap: `unique_ptr` is sole, move-only ownership; `shared_ptr` shares ownership through a reference count; `weak_ptr` observes without owning and breaks cycles. Raw pointers and references never own — they borrow. Default to `unique_ptr`, and remember the best pointer is often no pointer.

Activity 1 — Ownership Audit

DIFFICULTY: FOUNDATIONAL

Scenario: *Ownership mistakes are easiest to see on paper.*

Requirements:

- Take your Part 2 capstone (or any program using `new/delete`) and list every new, every delete, and every raw pointer, marking each as owning or borrowing.
- Replace every owning raw pointer with the right smart pointer, removing every delete.
- Run the result under AddressSanitizer and confirm there are no leaks.

Activity 2 — A Family Tree Without Leaks

DIFFICULTY: CORE

Scenario: *Parents and children refer to each other — a classic cycle.*

Requirements:

- Model people where parents own their children through `shared_ptr` and children refer to their parents.
- First make children hold `shared_ptr` to parents and prove (with destructor tracing) that the family leaks.
- Fix it with `weak_ptr`, and write a function that prints each person's parents only if they still exist.

Activity 3 — A Resource Cache

DIFFICULTY: CHALLENGE

Scenario: *Loading a large resource (an image, a configuration, a lookup table) twice is wasteful, but keeping it forever wastes memory.*

Requirements:

- Build a Cache that returns `std::shared_ptr<Resource>` to callers but stores only `std::weak_ptr` internally.
- If a resource is still in use somewhere, the cache returns the same object; once every user has released it, the next request loads it again.
- Demonstrate both behaviours with tracing, and explain why `weak_ptr` is essential to the design.

Domain Twist (optional)

EE twist: a Building where each Floor owns its Rooms (`unique_ptr`) but a central FireAlarm observes every room (`weak_ptr` or raw borrow - justify which). Biomedical

twist: a Theatre schedule that shares Surgeon objects between operations (shared_ptr) while each Operation owns its Instrument tray (unique_ptr).

Chapter 27 — Composition, Aggregation & Association

Recap: Composition: parts live and die with the whole (value members or `unique_ptr`). Aggregation: the whole groups parts that exist independently (non-owning pointers, references, `weak_ptr`). Association: peers that know each other — change two-way links through one method. Dependency: brief use through a parameter. Prefer composition over inheritance for reuse.

Activity 1 — Relationship Sorting

DIFFICULTY: FOUNDATIONAL

Scenario: *Classifying relationships is a skill that improves with practice.*

Requirements:

- For twenty pairs of classes you choose from everyday life and your track (e.g. Car/Wheel, Teacher/Student, Hospital/Ward), classify the relationship and justify it in one sentence based on lifetime.
- Draw the six most interesting ones in text UML with multiplicities.
- Implement three of them in C++ using the matching technique.

Activity 2 — Unpick an Inheritance Mess

DIFFICULTY: CORE

Scenario: *A previous developer built a menu system with `BaseMenu` -> `FileMenu` -> `FileMenuWithIcons` -> `FileMenuWithIconsAndShortcuts`, and the same chain for `EditMenu`.*

Requirements:

- Write a small version of that hierarchy (at least six classes) to feel the problem.
- Redesign it with composition: one Menu class that has optional icon and shortcut parts.
- Count classes before and after, and explain how adding a 'tooltips' feature differs in each design.

Activity 3 — A Consistent Many-to-Many Association

DIFFICULTY: CHALLENGE

Scenario: *Students take many courses; courses have many students. Both sides need to know.*

Requirements:

- Implement Student and Course where each keeps non-owning pointers to the other, with a single Enrolment manager that is the only way to enrol or withdraw.
- Prove consistency after a long random sequence of enrolments and withdrawals by checking both sides agree.
- Explain what would happen if a Course were destroyed while students still pointed at it, and change the design to make that impossible or detectable.

Domain Twist (optional)

EE twist: a Panel composed of Breakers, aggregating Circuits that can be moved between panels, with a two-way Circuit/Breaker association. Biomedical twist: a CareTeam aggregating Nurses with a two-way Nurse/Patient allocation kept consistent across shift changes.

Chapter 28 — Advanced Inheritance

Recap: Public inheritance means is-a; prefer composition to private inheritance. using Base::Base inherits constructors; declaring a name hides all base overloads unless you add using Base::name. final stops overriding or deriving. Slicing happens whenever a derived object is copied into a base object. A diamond needs virtual inheritance, and the most-derived class constructs the shared base.

Activity 1 — Name-Hiding Hunt

DIFFICULTY: FOUNDATIONAL

Scenario: Name hiding produces some of C++'s most confusing errors.

Requirements:

- Write three small hierarchies in which a derived class accidentally hides base overloads (one with different parameter types, one with const vs non-const, one with a default argument).
- Record each compiler error and fix each with a using-declaration.
- Explain in a comment why adding virtual would NOT have fixed any of them.

Activity 2 — Final Safety Rules

DIFFICULTY: CORE

Scenario: Some behaviour must never be changed by a subclass.

Requirements:

- Design a small hierarchy of safety devices in which one method (e.g. an emergency stop) is final and one leaf class is final.
- Write a subclass that attempts to override the final method and record the error.
- Write a paragraph arguing for or against making every leaf class final by default.

Activity 3 — Diamond, With and Without

DIFFICULTY: CHALLENGE

Scenario: The diamond problem is easiest to understand by building it both ways.

Requirements:

- Build a four-class diamond with data in the top class, once without and once with virtual inheritance; print sizeof each bottom class and the addresses of the top-class subobjects.
- Explain the differences you observe.
- Then redesign the same feature with composition and interfaces only, and compare the three versions for clarity.

Domain Twist (optional)

EE twist: a HybridInverter that is both a SolarInverter and a GridCharger sharing one Device base. Biomedical twist: a Defibrillator-Monitor that is both a Monitor and a Therapy device sharing one MedicalDevice base.

Chapter 29 — Abstract Classes, Interfaces & Advanced Polymorphism

Recap: A pure virtual function (`= 0`) makes a class abstract; an interface has only pure virtual functions and no data. Program to interfaces. Virtual calls go through a vtable; non-virtual calls are statically bound. Use `dynamic_cast` sparingly — a chain of type checks usually wants a virtual function. `clone()` copies polymorphic objects as their real type.

Activity 1 — Interfaces From Capabilities

DIFFICULTY: FOUNDATIONAL

Scenario: *Good interfaces describe capabilities, not things.*

Requirements:

- List ten capabilities from your track (e.g. Printable, Chargeable, Calibratable, Billable).
- Write four of them as interfaces, and three unrelated classes that each implement two of them.
- Write one function per interface that works with any implementer, and test each with all of its implementers.

Activity 2 — Replace the Type Switch

DIFFICULTY: CORE

Scenario: *Type checks with `dynamic_cast` or `typeid` spread through a codebase like weeds.*

Requirements:

- Write a program that calculates something (pay, energy, risk score) using a chain of `dynamic_cast`s over five derived types.
- Refactor it to use a virtual function, so the calculating function has no type checks at all.
- Add a sixth type to both versions and count how many places you had to edit in each.

Activity 3 — Polymorphic Undo Snapshots

DIFFICULTY: CHALLENGE

Scenario: *An editor holding a `vector<unique_ptr<Shape>>` wants to support 'undo' by saving snapshots.*

Requirements:

- Implement `clone()` for at least three shape types and use it to take a deep snapshot of the whole collection before each change.
- Implement undo by restoring the last snapshot, and prove with tracing that snapshots are independent of the live collection.

- Discuss in a comment the memory cost of full snapshots and one alternative design (hint: Module 34's Command pattern).

Domain Twist (optional)

EE twist: an abstract PowerSource (Grid, Solar, Generator) with a Template Method that prices each kWh. Biomedical twist: an abstract ClinicalScore (EWS, Glasgow Coma Scale, pain score) with a Template Method that validates inputs, scores and classifies.

Chapter 30 — Operator Overloading II & Friends

Recap: Write += as a member and + as a non-member built on it; provide symmetric non-member forms where either operand can come first. Prefix ++ returns *this; postfix takes a dummy int and returns the old value. Provide const and non-const []. Functors carry data into STL algorithms. >> must fail cleanly. Conversions should be explicit. <=> = default gives all comparisons in C++20.

Activity 1 — A Duration Type

DIFFICULTY: FOUNDATIONAL

Scenario: Times are often stored as bare ints of minutes, which invites mistakes.

Requirements:

- Build a Duration value type (stored in seconds) with +, -, +=, -=, scalar *, comparisons via <=>, and << printing h:mm:ss.
- Add a >> that reads h:mm:ss and fails cleanly on bad input.
- Write a judgement comment explaining why Duration * Duration is not provided.

Activity 2 — A Sparse Vector

DIFFICULTY: CORE

Scenario: Some vectors are mostly zeros — sensor grids, word counts, large timetables.

Requirements:

- Build a SparseVector storing only non-zero entries in a std::map<int, double>.
- Provide a const operator[] that returns 0 for missing entries, and a set(index, value) method instead of a non-const [] (explain why a non-const [] is tricky here).
- Overload + and dot product (a named function, not an operator — justify), and compare memory use with a dense vector.

Activity 3 — Functor Pipelines

DIFFICULTY: CHALLENGE

Scenario: Functors become powerful when they carry state across calls.

Requirements:

- Write three stateful functors: a MovingAverage(n), an OutlierFilter(k standard deviations), and a RateOfChange.
- Use them with std::transform and std::for_each to clean and analyse a series of 200 readings.
- Rewrite one of them as a lambda with a mutable capture, and explain which version you would keep in a real project.

Domain Twist (optional)

*EE twist: a Phasor type (magnitude and angle) with * and / in polar form and conversion to and from your Module 30 Impedance. Biomedical twist: a Concentration*

type (mg/mL) where Dose / Concentration gives a Volume — design the three types so only meaningful operations compile.

Chapter 31 — Exceptions in Class Design

Recap: Give each system one root exception class, derived from `std::runtime_error`, and exceptions that carry useful data. A throwing constructor means the object never existed — fully built members are destroyed, the destructor is not run. Destructors must not let exceptions escape. Aim for the strong guarantee; never fall below basic. RAII makes clean-up automatic during unwinding.

Activity 1 — Guarantee Labelling

DIFFICULTY: FOUNDATIONAL

Scenario: Every function gives some exception-safety guarantee, whether or not its author thought about it.

Requirements:

- Take a class you wrote earlier in Part 3 and label every public method no-throw, strong, basic or none, with a sentence of justification.
- Fix every method labelled 'none'.
- Upgrade at least one 'basic' method to 'strong', and write a test that proves it.

Activity 2 — An Error-Reporting Layer

DIFFICULTY: CORE

Scenario: Low-level errors (a failed `std::stoi`, a missing file) mean little to a user.

Requirements:

- Write a three-layer program: a parser that throws standard exceptions, a repository that translates them into your own hierarchy with context (file and line), and a user interface that prints friendly messages.
- Include one log-and-rethrow layer using `throw;`.
- Show that the original exception type is preserved through the rethrow but replaced at the translation boundary.

Activity 3 — A Transactional Container

DIFFICULTY: CHALLENGE

Scenario: Banks, schools and hospitals all need 'all or nothing' updates.

Requirements:

- Build a Ledger that supports `beginTransaction()`, several add/remove operations, and `commit()` or `rollback()`.
- Any exception during a transaction must leave the ledger exactly as it was before `beginTransaction()`.
- Test it with an operation that throws half-way through a batch, and explain your design in a short Markdown note.

Domain Twist (optional)

EE twist: a Switchboard whose `reconfigure()` either applies every breaker change or none, with an `OvervoltageError` carrying the measured voltage. *Biomedical twist:* a

BloodBank whose issueUnits() either reserves every unit for a transfusion or none, with a CrossMatchError carrying the incompatible blood groups.

Chapter 32 — Templates Meet OOP

Recap: Template member definitions live in headers. Member function templates, non-type parameters and specialisations make templates flexible. Templates and inheritance combine well: a class template derived from an abstract base can live in a polymorphic collection. Static polymorphism resolves at compile time; virtual functions at runtime. C++20 concepts state what a template needs.

Activity 1 — A Fixed-Size Matrix Template

DIFFICULTY: FOUNDATIONAL

Scenario: When matrix sizes are known at compile time, the compiler can check them for you.

Requirements:

- Build `Matrix<T, Rows, Cols>` with non-type parameters, storing a plain array.
- Provide multiplication that only compiles when the inner dimensions match (`Matrix<T, R, K> * Matrix<T, K, C>`).
- Show (in a comment) the error when multiplying incompatible sizes.

Activity 2 — Concept-Checked Algorithms

DIFFICULTY: CORE

Scenario: Generic algorithms should say what they need.

Requirements:

- Write concepts `Measurable` (has `value()` and `unit()`) and `Comparable` (supports `<`).
- Write constrained function templates `average()`, `maxBy()` and `report()` using them, and use them with two unrelated classes from your track.
- Deliberately misuse each template once and record the compiler message.

Activity 3 — Static vs Dynamic Shoot-Out

DIFFICULTY: CHALLENGE

Scenario: The same feature can be built with templates or with virtual functions.

Requirements:

- Implement a sensor-processing pipeline twice: once with an abstract base and virtual functions, once with templates (and optionally CRTP).
- Measure compile time, binary size and run time for 10 million readings in each version.
- Write a recommendation for which design your track's capstone should use, and why.

Domain Twist (optional)

EE twist: a `Quantity<Unit>` template so volts and amps cannot be added by mistake but volts x amps gives watts. Biomedical twist: a `Measurement<Vital>` template so a heart rate can never be passed where a temperature is expected.

Chapter 33 — SOLID Principles & Clean Design

Recap: Aim for high cohesion and low coupling. SRP: one reason to change. OCP: extend by adding code. LSP: subclasses keep the base's promises. ISP: small, focused interfaces. DIP: depend on abstractions and inject dependencies. Refactor in small, verified, committed steps without changing behaviour.

Activity 1 — Principle Flashcards From Real Code

DIFFICULTY: FOUNDATIONAL

Scenario: Principles stick when you've seen them in code you know.

Requirements:

- For each SOLID principle, find one place in your own Part 2 or Part 3 code that follows it and one that breaks it.
- Write each as a 'flashcard': the code snippet, the principle, and one sentence on the consequence.
- Fix the two violations you think matter most.

Activity 2 — Refactor With a Safety Net

DIFFICULTY: CORE

Scenario: Refactoring without a way to check behaviour is just editing and hoping.

Requirements:

- Pick a 200+ line program you wrote earlier in the course. Save its output for five different inputs.
- Refactor it towards SOLID in at least eight small commits, comparing the saved outputs after every commit.
- Write a REFACTORING.md listing each commit, the smell it removed and the principle it applied.

Activity 3 — Design Review Role-Play

DIFFICULTY: CHALLENGE

Scenario: Professional code goes through review before it's accepted.

Requirements:

- Swap a Part 3 project with a classmate. Write a structured review: three strengths, three SOLID concerns with suggested fixes, and one question.
- Respond in writing to the review you receive, accepting or rejecting each point with reasons.
- Implement the changes you accepted and note what you learned from defending the ones you rejected.

Domain Twist (optional)

EE twist: refactor a monolithic 'MeterReader' that reads, validates, prices, bills and emails into SOLID classes. Biomedical twist: refactor a monolithic 'LabResults' class that parses, flags, stores and notifies into SOLID classes.

Chapter 34 — Design Patterns in C++

Recap: Creational: Factory, Builder, Singleton (sparingly). Structural: Adapter, Composite, Decorator. Behavioural: Strategy, Observer, State, Command. Patterns are named solution shapes for recurring problems; use one when the problem matches, and name it so others understand your design.

Activity 1 — Pattern Spotting

DIFFICULTY: FOUNDATIONAL

Scenario: *Patterns are everywhere once you know their shapes.*

Requirements:

- Find at least five patterns in the C++ standard library or in code you already know (hint: iterators, `std::function`, stream manipulators, `std::make_unique`).
- For each, name the pattern, the roles, and the problem it solves.
- Write a short program using three of them deliberately.

Activity 2 — Undo/Redo Text Editor

DIFFICULTY: CORE

Scenario: *Undo and redo are the classic Command pattern application.*

Requirements:

- Build a tiny line-based text editor with insert, delete and replace commands, each with `execute()` and `undo()`.
- Support unlimited undo AND redo with two stacks.
- Add a 'macro' command that groups several commands into one (a Composite of Commands).

Activity 3 — A Plugin-Style Factory

DIFFICULTY: CHALLENGE

Scenario: *Large programs let new types register themselves without editing a central factory.*

Requirements:

- Build a factory registry where each concrete class registers itself (e.g. through a static registration object in its own .cpp file).
- Add three new types without editing the factory's source file.
- Explain the static initialisation order risk this introduces, and how you avoided it.

Domain Twist (optional)

EE twist: a traffic-light controller using State, with Observers for pedestrian buttons and a Strategy for day/night timing. Biomedical twist: a ventilator-mode controller using State, with Observers for alarms and a Strategy for adult/paediatric settings.

Chapter 35 — Testing & Debugging OO Code

Recap: Unit tests check one behaviour each, in Arrange-Act-Assert form, quickly and repeatably. Test boundaries, errors, exceptions and state after failure. Use fakes through interfaces to test in isolation. TDD: red, green, refactor. Use gdb to inspect, and sanitizers to catch memory errors as they happen.

Activity 1 — Test an Old Class

DIFFICULTY: FOUNDATIONAL

Scenario: *Code written without tests usually has bugs nobody has noticed.*

Requirements:

- Choose a class you wrote in Part 1 or Part 2. Write at least 20 checks covering normal cases, boundaries and invalid input.
- Record every bug your tests found.
- Fix them, committing the failing test before each fix.

Activity 2 — TDD Kata: Grade Calculator

DIFFICULTY: CORE

Scenario: *A kata is a small exercise repeated until the technique becomes natural.*

Requirements:

- Using strict TDD, build a WASSCE grade calculator (A1-F9) one test at a time, recording each red-green-refactor cycle in a log.
- Then start again from scratch and do it faster; compare your two logs.
- Write down which tests drove the most important design decisions.

Activity 3 — Mutation Testing by Hand

DIFFICULTY: CHALLENGE

Scenario: *How good is a test suite? One way to find out is to break the code on purpose.*

Requirements:

- Take a class with a good test suite. Make ten small deliberate 'mutations' (change < to <=, delete a line, flip a condition), one at a time.
- For each mutation, record whether your tests caught it.
- Add tests until every mutation is caught, and explain what kinds of tests you had been missing.

Domain Twist (optional)

EE twist: test a PID-controller class with a fake plant model that responds predictably. Biomedical twist: test an infusion-rate calculator with a fake clock and fake pump, including boundary doses and unit errors.

Extended Capstone — The Complete Architecture

Module 36's own capstone already brings every Part 3 concept into one staged, tested system. This Extended Capstone is optional and goes further: your system gains a second, independent front end, a plug-in mechanism for new types, data migration between file-format versions, and a written architecture review. Treat it as a portfolio centrepiece, not a required assignment — attempt it once your Module 36 capstone is complete and all of its tests pass.

Extend the same track you used for Module 36 (Generic, EE or Biomedical). All three tracks share the structural requirements below, applied to their own domain.

1. A second front end (Model-View-Controller)

Separate your system into Model, View and Controller (Module 34). Keep the existing console menu as one View, and add a second, completely different View — for example a report generator that writes an HTML dashboard file, or a scripted 'batch' interface that reads commands from a text file. Neither View may contain business rules; both must work against the same Model, with no changes to the Model's code when the second View is added.

2. Plug-in types

Make your main hierarchy extensible without editing existing files: new types (a new device, assessment or monitor) register themselves with a factory registry (Module 34's Challenge activity). Add two new types this way, and list in your write-up every existing file you did - or did not - have to touch.

3. Versioned data and migration

Change your data file format (add a field, rename a column, split a file) and give each file a version number on its first line. Your loader must read BOTH the old and new versions, migrating old data to the new model with the strong guarantee: a migration that fails part-way leaves the old files and the in-memory system untouched.

4. A continuous-integration-ready test suite

Grow your test suite to at least 80 checks, including tests for both Views (using fakes for file output), the plug-in registry, and the migration (including a corrupted old-version file). Make the whole suite run with one command through ctest with sanitizers enabled, ready to be run automatically on every push.

5. An architecture review (ARCHITECTURE.md)

Write 600-1000 words, with diagrams, covering: the layers of your system and the dependency direction between them (Module 33's DIP); each pattern and the change it made easy; one design decision you reversed during the project and why; and a risk register of the three parts of the system most likely to cause problems as it grows, with how you would mitigate each.

When You're Done

This project is intentionally open-ended — there is no single correct architecture. Grade yourself (or ask your instructor to) against whether the Model is genuinely independent of both Views, new types and file versions can be added with minimal edits to existing code, the system is exception-safe and sanitizer-clean, and ARCHITECTURE.md shows specific, honest thinking rather than generic statements. Commit it as its own folder, e.g. `project-book/extended-capstone-3/`, with many small commits that tell the story of how it was built.